

Guia de testes entre máquinas: macOS + Windows

T1 Laboratório de Redes · Servidor HTTP/1.1 sobre sockets TCP · Grupo 5

Papéis: o **Mac** é o **servidor** e faz as capturas (`tshark` + `metricas.py` já validados nele). O **Windows** é o **cliente** (navegador, `curl` e scripts). Em todo o guia, troque **<IP_MAC>** pelo IP do Mac descoberto na Fase 1.

Hotspot do iPhone = só IPv6. O **<IP_MAC>** é um endereço IPv6 (ex.: `2804:18:::fcb2`). Nos **scripts** ele vai puro; em **URLs** (`curl`, navegador) vai **entre colchetes**: `http://[<IP_MAC>]:8080/`.

MAC comando no Terminal do Mac **WINDOWS** comando no **Git Bash** do Windows (salvo indicação contrária)

Por que usar o Git Bash no Windows?

Os scripts de teste do projeto (`scripts/*.sh`) são **scripts bash**. O Windows não tem bash nativo: o **PowerShell** e o **cmd** são outras linguagens de shell e não conseguem executá-los. O Git Bash resolve isso:

- **Executa os `.sh` sem alteração.** Eles usam recursos próprios do bash: `$'\n\r\n...'` (para montar o cabeçalho malformado do teste 400), arrays (`args+=(...)` no C2), `time`, `for ... in $(seq 10)` e heredocs. Nada disso existe no PowerShell/cmd.
- **Traz o `curl` de verdade.** No PowerShell, `curl` é um *alias* para `Invoke-WebRequest`, que tem outra sintaxe e não aceita `-i`, `--path-as-is` ou `--request-target`, usados nos testes de 400 e 403. No Git Bash, `curl` é o curl real (no PowerShell seria preciso digitar `curl.exe`).
- **Tem as ferramentas Unix** que os scripts chamam (`seq`, `grep`, `/dev/null`). Assim o **mesmo comando** funciona no Mac, no Ubuntu e no Windows, e o relatório mostra comandos idênticos.
- **É leve e não mexe na rede.** A alternativa, o WSL, instala um Linux inteiro. No WSL2 ele fica atrás de uma NAT com IP próprio: como cliente funciona, mas **não serve para rodar o servidor** (as outras máquinas não o alcançam).
- **Vem junto com o Git**, que o grupo já usa para trocar o código.

Dentro do Git Bash, `ping` continua sendo o ping do Windows: use `-n` (não `-c`) para o número de pacotes.

Fase 0: Preparação (uma vez)

WINDOWS Instalar

1. **Git for Windows** (git-scm.com/download/win): instala o Git Bash.
2. **Python 3.9 ou superior** (python.org), marcando “*Add python.exe to PATH*”. Só é necessário para o `simultaneo.sh` e para a Fase 10.
3. Receber o projeto (zip ou repositório privado no GitHub) e entrar na pasta:

```
cd ~/Downloads/T1-lab
```

AMBOS Mesma rede Wi-Fi, **VPN desligada**.

Fase 1: Verificação inicial (antes de tudo)

MAC Descobrir o IPv6 fixo (o endereço “temporary” muda sozinho; não use):

```
ifconfig en0 | grep "autoconf secured" | grep -v deprecated | awk '{print $2}'
```

WINDOWS Testar o alcance até o Mac por IPv6:

```
ping -6 -n 4 <IP_MAC>
```

Rede com IPv4 (Wi-Fi comum, laboratório): o IP sai de `ipconfig getifaddr en0`, o ping é `ping -n 4 <IP_MAC>` e as URLs **não** levam colchetes. O resto do guia é igual.

O IPv6 do Mac pode **mudar quando o hotspot reconecta**: se algo parar de funcionar, rode de novo o comando acima. Sem resposta no ping? A rede isola os clientes (comum em redes de campus). O ping **Mac → Windows** costuma falhar, porque o Windows bloqueia ping de entrada por padrão. Isso é normal e não atrapalha, já que o servidor é o Mac. Se as máquinas não se alcançarem na aula, avisem o professor **imediatamente**.

Fase 2: Subir o servidor

MAC Iniciar o servidor salvando o log (evidência para o item 5):

```
cd ~/Developer/T1-lab && python3 server.py --port 8080 --root ./www --host :: | tee captures/servidor.log
```

`--host ::` faz o servidor escutar em IPv6 e IPv4 no mesmo socket (dual-stack). Sem ele, o padrão é `0.0.0.0`, só IPv4.

Se o macOS perguntar “*Deseja permitir que python3 aceite conexões de entrada?*”, clique em **Permitir**. Deixe esse terminal aberto e use **outro** para as capturas.

WINDOWS Teste rápido:

```
curl -i "http://[<IP_MAC>]:8080/"
```

Deve vir `HTTP/1.1 200 OK`, e o log do Mac deve mostrar o IP do Windows.

Fase 3: Navegador (ensaio da interoperabilidade)

WINDOWS No navegador, abra `http://[<IP_MAC>]:8080/` (com os colchetes). As duas imagens devem aparecer e as linhas “JavaScript carregado” e “JSON carregado” devem ficar **verdes**. Tire um print. No log do Mac aparecem as conexões paralelas do navegador e, 5 s depois, o fechamento por timeout.

Fase 4: Conformidade e travessia (itens 2 e 3 do relatório)

WINDOWS

```
scripts/conformidade.sh <IP_MAC> > conformidade.txt
```

```
scripts/travessia.sh <IP_MAC> > travessia.txt
```

Conferir: `conformidade.txt` com 200, 200, 400, 400, 403, 404, 405, 405; `travessia.txt` com cinco 403. Esse texto vai direto para as tabelas do relatório.

Fase 5: Captura de uma transação completa (item 4)

MAC Iniciar a captura:

```
tshark -i en0 -f "tcp port 8080" -w capturas/transacao.pcapng
```

WINDOWS

```
curl -s -o /dev/null "http://[<IP_MAC>]:8080/index.html"
```

MAC Esperar 2 s e parar com **Ctrl+C**. Abrir no Wireshark e marcar no print: handshake (SYN, SYN-ACK, ACK), o pacote com o `GET`, os pacotes da resposta e o encerramento (FINs).

Fase 6: Atendimento simultâneo (item 5)

O enunciado pede **duas máquinas clientes ao mesmo tempo**, e o Mac é o servidor. O segundo cliente é a máquina do colega do **Ubuntu**. Se ele não estiver disponível, serve um **celular** na mesma Wi-Fi abrindo `http://[<IP_MAC>]:8080/`.

WINDOWS + Ubuntu, **ao mesmo tempo**:

```
scripts/simultaneo.sh <IP_MAC>
```

O `capturas/servidor.log` do Mac mostra conexões de dois IPs intercaladas, em threads diferentes. Copiar o trecho para o relatório.

Fase 7: RTT (item 6)

WINDOWS Logo antes das medições C1/C2 (na Wi-Fi o RTT varia):

```
ping -6 -n 20 <IP_MAC>
```

Anotar a **Média** exibida no final.

Fase 8: Medições C1 e C2 (itens 7 a 9), 3 rodadas de cada

Para cada rodada `N` = 1, 2, 3:

C1: uma conexão nova por requisição

MAC

```
tshark -i en0 -f "tcp port 8080" -w capturas/c1_N.pcapng
```

WINDOWS

```
scripts/c1.sh <IP_MAC>
```

MAC Quando o script terminar, esperar 2 s e dar **Ctrl+C**.

C2: uma única conexão persistente

MAC

```
tshark -i en0 -f "tcp port 8080" -w capturas/c2_N.pcapng
```

WINDOWS

```
scripts/c2.sh <IP_MAC>
```

MAC Esperar 2 s e dar **Ctrl+C**.

Durante as medições **feche o navegador** que estiver aberto na página do servidor: as conexões dele entrariam na captura. Conferência: o `c1.sh` mostra `conexões novas 1` em todas as linhas; o `c2.sh` só na primeira (as demais mostram 0).

Fase 9: Extrair as métricas

MAC

```
for f in capturas/c1_*.pcapng capturas/c2_*.pcapng; do python3 scripts/metricas.py "$f"; done
```

Escolher a rodada **mediana** de cada cenário e renomear para `capturas/c1.pcapng` e `capturas/c2.pcapng` (nomes pedidos na entrega). As outras rodadas ficam fora do zip.

Fase 10 (opcional): Inverter os papéis

Ensaia o caso em que o servidor do grupo roda no Windows durante a apresentação.

1. **WINDOWS** **Configurações** → **Rede e Internet** → **Wi-Fi** → **(sua rede)** → marcar **Rede privada**. Em rede Pública o firewall bloqueia conexões de entrada.
2. **WINDOWS** No **PowerShell** (aqui não precisa de Git Bash; é só o Python):

```
py server.py --port 8080 --root .\www
```

3. Aceitar o aviso do Firewall do Windows marcando **Redes privadas**.
4. **WINDOWS** Descobrir o IP com `ipconfig` (linha “Endereço IPv4” do adaptador Wi-Fi).
5. **MAC** Abrir no navegador `http://<IP_WINDOWS>:8080/`.

Não rode o servidor dentro do WSL2: ele fica atrás de uma NAT e as outras máquinas não o alcançam.

Se algo falhar

Sintoma	Causa provável / solução
<code>ping</code> funciona, mas o <code>curl</code> dá <i>Connection refused</i>	Servidor parado, porta errada ou servidor sem <code>--host ::</code> (escutando só IPv4).
<code>curl : URL rejected / bad range</code> com IPv6	Faltaram os colchetes: <code>"http://[2804:...]:8080/"</code> (use aspas). Em curl antigo, acrescente <code>-g</code> .
<code>curl</code> fica parado até dar <i>timeout</i>	Firewall do Mac bloqueando o Python: Ajustes do Sistema → Rede → Firewall → Opções → permitir python3.
O <code>tshark</code> do Mac não captura nada	Interface errada: rodar <code>tshark -D</code> e usar a da Wi-Fi.
<code>\r: command not found</code> ao rodar um <code>.sh</code> no Windows	Scripts convertidos para CRLF. Usar o repositório com <code>.gitattributes</code> ou rodar <code>dos2unix scripts/*.sh</code> no Git Bash.
No PowerShell, <code>curl -i</code> dá erro de parâmetro	<code>curl</code> é alias de <code>Invoke-WebRequest</code> . Use o Git Bash ou <code>curl.exe</code> .
<code>python3</code> abre a Microsoft Store no Windows	É só um atalho. Use <code>py</code> (o <code>simultaneo.sh</code> já detecta sozinho).

Observação para o relatório (IPv6)

Como as medições são em IPv6, cada pacote carrega um cabeçalho IP de **40 bytes** (contra 20 no IPv4). Isso aumenta o custo em bytes de cada handshake e encerramento; vale citar na análise do overhead (item 8). No Wireshark, use o filtro `tcp.port == 8080`, que funciona igual para IPv4 e IPv6.

Checklist de evidências para o relatório

Item	Evidência	Fase
2 – Conformidade	<code>conformidade.txt</code>	4
3 – Travessia	<code>travessia.txt</code>	4
4 – Transação completa	<code>capturas/transacao.pcapng</code> + print do Wireshark anotado	5
5 – Atendimento simultâneo	trecho de <code>capturas/servidor.log</code> com dois IPs	6
6 – RTT	saída do <code>ping -6 -n 20</code>	7
7 a 9 – C1 × C2	<code>capturas/c1.pcapng</code> , <code>capturas/c2.pcapng</code> + saída do <code>metricas.py</code>	8 e 9
Interoperabilidade	print do navegador com a página completa	3